

1. What is Agentic Coding?

Agentic Coding is a software development approach in which an AI agent can do more than simply generate code from a user's instructions. The agent can understand a software project, inspect existing files, plan a solution, modify multiple files, execute commands, run tests, analyze errors, and continue working toward a specified goal.

In traditional AI-assisted coding, the developer usually gives a prompt and receives a code suggestion. In agentic coding, the AI operates more like an active development assistant. It can make a plan, take several actions, observe the results, and adjust its approach when necessary.

For example, if a developer asks an agent to “add user authentication to this Django application,” an agentic coding system may inspect the existing project structure, identify the relevant models, views, URLs, and templates, implement authentication, update configuration, run tests, and fix problems discovered during testing.

Therefore, agentic coding can be described as goal-oriented AI-driven software development where the AI is capable of reasoning about a codebase and taking multiple actions to complete a development task.

2. How is AI-Assisted Coding different from Agentic Coding?

AI-Assisted Coding and Agentic Coding both use artificial intelligence to support software development, but they differ mainly in the level of autonomy given to the AI.

AI-Assisted Coding generally works as a developer-support tool. The developer asks the AI to generate a function, explain an error, complete code, or suggest an implementation. The developer remains responsible for deciding what should be changed and usually applies or reviews the generated code manually.

Agentic Coding is more autonomous and goal-oriented. Instead of only generating suggestions, an AI coding agent can inspect the project, create a plan, edit files, execute commands, run tests, examine the results, and make additional changes when required.

The major differences are:

- **Level of autonomy:** AI-assisted coding has lower autonomy, while agentic coding can perform multiple development actions independently.
- **Scope of work:** AI-assisted coding often focuses on individual code snippets or specific problems, whereas agentic coding can work across an entire project.
- **Iteration:** AI-assisted coding commonly follows a prompt-response process. Agentic coding can follow a continuous cycle of planning, acting, testing, observing, and correcting.
- **Developer involvement:** In AI-assisted coding, the developer generally directs and applies the changes. In agentic coding, the developer provides a goal and supervises the agent's work.

In short, AI-assisted coding is primarily AI helping the programmer write code, while agentic coding is AI acting as a more autonomous development agent that can work toward completing a software task.

3. What is Claude Code used for?

Claude Code is an AI-powered coding agent developed by Anthropic. It is designed to work with a software project through the terminal and can help developers understand, modify, test, and maintain code.

Claude Code can be used for several software development activities:

1. Understanding codebases: It can inspect project files and help explain how different parts of an application work.
2. Writing and modifying code: It can implement new features, create or update files, and make changes across multiple parts of a project.
3. Debugging: It can investigate errors, trace possible causes, modify the relevant code, and help verify the solution.
4. Refactoring: It can improve the structure and readability of existing code while attempting to preserve its intended behavior.
5. Testing: It can run existing tests, analyze test failures, and make appropriate changes based on the results.
6. Working with development tools: Because it operates in the project environment, it can use commands and development tools available to the project.

For example, a developer working on a Django project could ask Claude Code to inspect an API endpoint, identify why a test is failing, modify the relevant view or serializer, run the tests again, and report the result.

Thus, Claude Code is useful for accelerating software development tasks that require understanding and interacting with an existing codebase rather than only generating isolated code snippets.

4. How can Claude Code help debug and refactor code?

Claude Code can support debugging and refactoring by interacting with the codebase, analyzing the existing implementation, executing relevant commands, and iterating based on the results.

Debugging process:

1. Identify the problem: The developer provides an error message, failing test, unexpected behavior, or a description of the problem.
2. Inspect the codebase: Claude Code can locate relevant files and examine the surrounding code to understand how the problem occurs.
3. Analyze the possible cause: It can trace the flow of data and identify suspicious logic, incorrect assumptions, missing conditions, or configuration problems.

4. Apply a fix: It can modify the relevant code instead of only describing a possible solution.
5. Run tests or commands: It can execute appropriate tests or development commands to check whether the change resolves the problem.
6. Iterate if necessary: If the tests still fail, the agent can examine the new results and make further changes.

For refactoring, Claude Code can identify duplicated logic, overly complex functions, poor organization, or code that can be made clearer. It can then reorganize or rewrite the code while attempting to maintain the existing functionality.

For example, if a Django application contains duplicated validation logic in several views, Claude Code may identify the duplication, suggest a reusable function or service, update the affected files, and run tests to verify that the refactoring did not break existing behavior.

However, developers should review the changes because an AI agent can misunderstand business requirements or introduce unintended changes. Automated testing and human review are therefore important parts of the process.

5. Why are testing and validation important when using AI coding agents?

Testing and validation are essential when using AI coding agents because AI-generated code is not guaranteed to be correct, secure, efficient, or consistent with the project's requirements.

AI coding agents can produce code that looks reasonable but may contain logical errors, incorrect assumptions, security vulnerabilities, compatibility problems, or unintended side effects. An agent may also misunderstand a requirement and implement something that technically works but does not satisfy the actual business need.

Testing helps detect these problems. Unit tests can verify individual functions or components, integration tests can verify that different parts of a system work together, and end-to-end tests can check important user workflows. Running tests after AI-generated changes provides evidence about whether the software continues to behave correctly.

Validation is also important beyond automated tests. Developers should review the generated code for correctness, security, performance, maintainability, and compliance with project requirements. In some cases, tests may pass even though the implementation is not appropriate for the intended use.

A useful workflow is:

Requirement → AI Agent → Code Changes → Testing → Validation → Human Review → Deployment

Therefore, testing and validation act as safeguards when using AI coding agents. They allow developers to benefit from AI's speed and automation while maintaining software quality, reliability, and security.